

OrangeOS v1.0

5分钟答辩演示 与操作系统课程问答

演示时间轴 | 逐字讲稿 | 32位分页 | 进程与调度 | 中断与系统调用 | 文件系统
| 高频追问

项目路径: D:\Oranges

答辩准备资料

阅读导航

第一部分用于现场演示，建议完整计时练习三遍；第二部分用于回答操作系统课程问题，先掌握每节的“标准回答”，再看追问。

部分	内容	用途
一	5分钟演示总流程与命令速记	现场操作
二	分阶段逐字讲稿	控制讲述节奏
三	操作系统课程视角描述	回答项目定位
四	32位内存与页面管理	重点原理题
五	中断、异常、调度、Ring 3与系统调用	核心机制题
六	设备、文件系统、交互和测试	系统综合题
七	高频短问短答与应急方案	临场复习

第一部分 5分钟演示总流程

演示主线：启动 → 自检 → 可观测调度 → OrangeFS加载Ring 3程序 → 文件持久化 → GUI集成 → 自动回归。不要追求命令数量，要让每个操作都证明一个课程知识点。

时间	演示	操作	核心证明
0:00-0:35	定位与启动	启动QEMU	裸机引导到32位内核
0:35-1:05	综合自检	selftest	关键模块处于可用状态
1:05-1:40	系统监视	monitor, 约5秒后Q	PIT、内存、抢占调度
1:40-2:25	用户态程序	exec demo.oex; ps	FS→分页→Ring3→系统调用
2:25-3:25	文件系统	stat big.txt; cat defense.txt; disk	跨扇区、持久化、位图
3:25-4:20	桌面集成	gui; 进入Files; 打开文件; 退出	VGA、键盘焦点、OrangeFS复用
4:20-5:00	回归与总结	展示16/16结果	端到端工程验证

现场命令速记

```
selftest
monitor → Q
exec demo.oex
ps
stat big.txt
cat defense.txt
disk
gui → Files → 打开文件 → Esc → Q → Q
```

演示前准备

- 提前创建并写入 defense.txt，重启一次确认仍可 cat，现场用它证明持久化。
- 提前运行 make test-release，保留 PASSED (16/16) 的终端或截图；现场不要等待整套回归。
- QEMU窗口保持焦点，关闭输入法中文模式，确认 Q、Esc、F2 和方向键可用。
- 用手机计时，目标在4分35秒左右讲完，留20秒处理切窗或输入失误。

第二部分 5分钟逐字讲稿

0:00-0:35 项目定位与启动

老师好，我的项目是 OrangeOS，它是运行在 x86 32位保护模式下的教学型单体内核操作系统。系统不是运行在 Windows 上的普通应用，而是从 BIOS 引导开始，通过 Bootloader 和 Loader 装载内核、建立 GDT 并进入保护模式，随后初始化中断、分页、内存、任务、系统调用、ATA 和 OrangeFS，最终进入现在看到的 Shell。出现 OrangeOS 提示符，说明从引导扇区到32位内核入口的完整启动链已经完成。

0:35-1:05 selftest

我首先运行系统内部自检。CPU 和 PG 表示保护模式与分页已开启；IDT 表示中断表已加载；MEM 和 HEAP 表示物理页与内核堆可用；TASK 表示时钟已经驱动任务调度；INT80 表示系统调用入口可用；ATA 和 FS 表示磁盘与文件系统可读；RING3 表示用户态环境已经准备完成。它属于运行时快速检查，后面还有宿主机驱动 QEMU 的端到端回归。

1:05-1:40 monitor

这里是实时系统监视器，可以看到 uptime、物理页使用量、调度切换次数、当前 PID，以及各任务的状态和累计运行 tick。PIT 约每10毫秒产生一次 IRQ0，调度器以10个 tick 左右作为时间片，因此约100毫秒可以发生一次抢占切换。监视器每10个 tick 刷新一次，空闲时使用 hlt 等待中断。Q键由键盘中断设置退出标志，实际退出由主循环完成。

1:40-2:25 exec demo.oex 与 ps

接下来展示一条跨模块完整执行链。demo.oex 保存在 OrangeFS 中，exec 先通过 ATA 读取并验证文件，再为它分配用户代码页、数据页和栈页，构造 PID 4 的初始现场，并由调度器使其以 Ring 3 权限运行。现在显示的文字不是用户程序直接操作显存，而是通过 int 0x80 请求内核输出。系统调用时 CPU 根据 TSS 切到可信 Ring 0 栈，内核验证用户指针和长度后执行服务。ps 显示 PID 4 已退出，代码、数据和栈三页随后被解除映射并释放。

2:25-3:25 OrangeFS

OrangeFS 使用固定布局：LBA 82 是主目录，83到114是32扇区数据池，115是备份目录。big.txt 大小700字节，占两个连续扇区，用来验证512字节边界上的跨扇区读写。cat 会按目录项读取数据并重新计算文件校验。defense.txt 是此前写入并经过重启后保留的文件，证明数据不是只在内存中，而是经 ATA 持久化到磁盘镜像。disk 根据32位空间位图统计空闲扇区。目录还具有主备副本、版本号 and XOR 校验，可恢复单份目录损坏。

3:25-4:20 GUI与文件管理器

这是直接写入 0xB8000 显存实现的 VGA 彩色文本桌面，不依赖宿主窗口库。它以状态机管理桌面、监视器、文件管理器、编辑器和小游戏。文件管理器直接复用 OrangeFS 接口，打开、修改、保存和删除都是真实磁盘操作。键盘中断只设置动作和重绘标志，耗时的文件读写与全屏绘制由主循环完成。

4:20-5:00 自动回归与结论

项目还建立了16项 QEMU 自动回归。宿主机通过 sendkey 模拟真实键盘输入，因此命令仍经过 IRQ1、键盘驱动和 Shell；系统通过 0xE9 端口输出日志。测试覆盖启动、终端、GUI、文件 CRUD、Tetris、Ring 3、进程生命周期、用户故障隔离、页面回收、OEX2加载及文件系统故障恢复。总体上，OrangeOS 把课程中的启动、资源管理、保护、抽象、交互和测试连接成了一个可运行、可持久化、可重复验证的系统。我的演示结束，谢谢老师。

第三部分 从操作系统课程角度描述 OrangeOS

标准定义

OrangeOS 是一个运行在 x86 32位保护模式下、以教学实验为目的的单体内核操作系统。它从 BIOS 引导开始，自主完成内核装载、硬件初始化、物理与虚拟内存管理、中断和异常处理、抢占式任务调度、Ring 3隔离、受控系统调用、ATA磁盘驱动与持久化文件系统，并通过 Shell、系统监视器和 VGA 文本应用提供交互，通过 QEMU 回归验证关键执行链。

课程职责	OrangeOS实现	课程关键词
处理器管理	PIT时钟、PCB、上下文切换、轮转	抢占、时间片、状态
内存管理	物理页、两级分页、内核堆	分配、映射、保护
中断与异常	IDT、PIC、IRQ、#PF	事件、现场、隔离
用户/内核边界	Ring0/Ring3、TSS、int 0x80	特权级、可信入口
设备管理	VGA、键盘、PIT、RTC、ATA	驱动、端口、IRQ
文件管理	OrangeFS、位图、目录项、校验恢复	持久化、抽象、一致性
系统交互	Shell、监视器、桌面和应用	接口、可观测性
工程质量	selftest与16项QEMU回归	可重复、防回归

内核结构

主要驱动、调度、内存、文件系统、Shell和GUI都在 Ring 0 共享地址空间并通过函数调用协作，因此属于单体内核。优点是结构直接、调用开销小、适合教学；代价是内核模块间隔离较弱，一个内核错误可能影响整个系统。只有专门加载的 OEX2 用户程序运行在 Ring 3。

完整技术链

BIOS → Bootloader → Loader → 保护模式 → GDT/TSS → IDT/PIC/PIT → 物理页与分页 → 抢占调度 → Ring 3与int 0x80 → ATA与OrangeFS → Shell/GUI → QEMU回归

第四部分 32位页面管理与内存问题

4.1 32位地址空间如何理解

32位地址最多表示 2^{32} 个字节，即4 GiB虚拟地址空间。开启分页后，程序使用的线性地址不一定等于物理地址，CPU通过页目录和页表完成转换。OrangeOS使用4 KiB页面，因此一个页面的页内偏移需要12位。

32位线性地址：页目录索引10位 | 页表索引10位 | 页内偏移12位

部分	位数	范围	作用
页目录索引	10	0-1023	选择PDE
页表索引	10	0-1023	选择PTE
页内偏移	12	0-4095	定位页内字节

一张页表有1024个PTE，每项映射4 KiB，所以一张页表覆盖4 MiB；一个页目录有1024个PDE，因此理论上覆盖 $1024 \times 4 \text{ MiB} = 4 \text{ GiB}$ 。

4.2 页面是怎样分配的

页面分配分成两个不同动作：第一步由物理页分配器选择一个空闲的4 KiB物理页并标记占用；第二步由分页模块在PDE/PTE中建立“虚拟页→物理页”映射。只分配物理页而不映射，程序仍无法用目标虚拟地址访问；只建立映射而不管理物理页，可能造成两个对象误用同一页。

申请用户页 → 物理页分配器找到空闲页 → 设置位图/占用状态 → 找到对应PDE/PTE → 写入物理页基址和权限 → invlpg刷新TLB

4.3 为什么每页4 KiB

4 KiB是32位x86普通分页的基本页大小。它在页表开销和内部碎片之间折中：页面太小会增加页表项和TLB压力，页面太大会增加小对象占用的内部碎片。

4.4 用户程序三页

虚拟地址	用途	权限
0x40000000	用户代码页	Present/Writable/User
0x40001000	用户数据页	Present/Writable/User
0x40002000	用户栈页，栈顶0x40003000	Present/Writable/User

用户任务运行时物理页使用量呈 $N \rightarrow N+3 \rightarrow N$ ：创建时增加代码、数据、栈三页，正常退出或用户页故障后解除映射并归还三页。

4.5 页表项权限

位	名称	含义
P	Present	为0时访问触发页故障
R/W	Read/Write	控制是否允许写
U/S	User/Supervisor	为1才允许Ring3访问

用户访问要成功，相关PDE和PTE都必须允许User；任意一级限制为Supervisor，Ring 3访问都会失败。项目内核映射常用0x03，用户映射使用0x07。

4.6 CR0、CR2、CR3与TLB

对象	职责
CR0.PE	开启保护模式
CR0.PG	开启分页
CR3	保存当前页目录物理地址，也关联当前地址空间
CR2	页故障时保存导致故障的线性地址
TLB	缓存近期虚拟页到物理页的转换

页表位于内存，但CPU不会每次都完整查询，因此使用TLB缓存。修改映射后执行 `invlpg`，使该虚拟页旧缓存失效，否则CPU可能继续使用旧映射。

问：物理地址、线性地址和虚拟地址有什么区别？

答：在当前32位平坦分段设计中，逻辑地址经段基址形成线性地址，段基址通常为0，所以程序看到的虚拟/线性地址数值相同；分页再把线性地址转换为物理地址。

问：分页和分段分别解决什么？

答：GDT分段建立段属性和特权级；分页负责页级地址转换和权限隔离。项目采用平坦分段，把细粒度内存管理主要交给分页。

问：为什么用户代码页当前也是可写的？

答：项目使用0x07统一映射三页以简化教学实现。更严格的系统应在装载后把代码页设为只读/可执行、数据和栈设为可写，并考虑NX等机制。

问：是否每个进程都有独立页目录？

答：当前是教学型简化设计，用户程序使用固定虚拟地址和共享式分页框架，没有实现现代系统完整的每进程独立地址空间。改进方向是每进程维护CR3和独立用户映射，同时共享内核高地址映射。

第五部分 中断、调度、Ring 3与系统调用

5.1 GDT、IDT、PIC和PIT关系

机制	回答的问题
GDT/TSS	当前以什么特权运行，跨级时使用哪一个内核栈
IDT	某个向量发生后跳到哪个处理程序
PIC	外部IRQ怎样屏蔽、排序并映射为向量
PIT	怎样周期性产生IRQ0时钟节拍

PIT到点 → IRQ0 → PIC映射为0x20 → IDT[0x20] → 时钟入口 → 调度器

CPU异常不经过PIC，例如除零和页故障由CPU直接按向量查询IDT。PIC重映射到0x20开始，是为了避开CPU保留的0-31号异常。硬件IRQ处理结束需要发送EOI。

5.2 抢占式调度

IRQ0入口pushad保存通用寄存器，调度器把当前ESP保存到PCB。时间片到期后，将当前任务标为READY，按轮转方式查找下一个READY任务，将其标为RUNNING并返回保存的ESP。切换ESP后，popad和iretd从另一任务的栈恢复现场。

IRQ0保存现场 → PCB.current.ESP=当前ESP → 轮转选择READY → ESP=next.ESP → popad → iretd

问：为什么切换ESP就相当于切换任务？

答：每个任务完整的寄存器和中断返回帧都保存在自己的内核栈上，ESP决定后续从哪份现场恢复。

问：抢占式和协作式调度有什么区别？

答：协作式依赖任务主动让出CPU；抢占式通过时钟中断强制夺回控制权，单个任务无法无限期独占处理器。

问：为什么中断入口要保存寄存器？

答：恢复后任务必须看到与中断前一致的通用寄存器状态，调度器也需要完整现场支持上下文切换。

5.3 Ring 0与Ring 3

Ring 0拥有最高权限，用于内核和驱动；Ring 3权限最低，用于用户程序。用户代码选择子0x1B、用户数据/栈选择子0x23的低两位为3。内核构造包含EIP、CS、EFLAGS、ESP、SS的返回帧，通过iretd进入Ring 3。

5.4 TSS与系统调用

用户执行int 0x80时，IDT门DPL=3允许主动进入。发生跨特权级切换时，CPU从TSS读取SS0/ESP0，换到可信内核栈。EAX保存调用号，内核分派有限服务，完成后iretd返回。

Ring3 int 0x80 → IDT[0x80] DPL3 → TSS切Ring0栈 → 保存现场 → 校验参数 → 执行 → EAX返回值 → iretd

问：为什么用户不能直接call内核函数？

答：普通call不能合法改变特权级，也不能建立可信内核栈和统一的参数检查边界；系统调用门提供受CPU保护的入口。

问：为什么0x80门的DPL必须为3？

答：软件int指令会检查门DPL。若门只允许DPL0，Ring3主动执行int 0x80会触发一般保护异常。

问：TSS是否负责项目中的任务调度？

答：不是。调度器使用软件保存PCB和ESP；TSS主要为Ring3进入Ring0提供SS0和ESP0。

5.5 用户指针校验

SYS_WRITE检查调用来源、非零长度、最大127字节、地址加长度溢出，以及缓冲区是否完整位于合法用户代码/数据/栈区。通过后先复制到内核缓冲区再输出。系统调用门只解决“允许进入”，参数校验才解决“进入后不能滥用”。

5.6 页故障隔离

CPU把故障地址写入CR2并压入错误码。处理程序检查旧CS低两位：Ring3故障标记当前任务FAULTED、记录CR2/错误码/EIP、释放三页并切换其他任务；Ring0故障则打印诊断并停机，因为内核状态已不可信。

第六部分 设备、文件系统、交互与测试

6.1 设备管理

设备	访问方式	用途
VGA文本	写0xB8000	Shell、监视器、文本桌面
键盘	IRQ1与扫描码	输入及应用焦点
PIT	端口配置与IRQ0	计时、调度、刷新、游戏
RTC	CMOS端口	日期时间
ATA	PIO、LBA28、0x1F0-0x1F7	512 B扇区读写

问：轮询和中断驱动有什么区别？

答：轮询由CPU反复检查设备状态；中断由设备在事件发生时通知CPU。键盘使用中断，ATA PIO数据传输前则通过状态端口轮询BSY/DRQ。

问：为什么GUI不在IRQ1里直接保存文件？

答：磁盘写入和重绘耗时。IRQ1只设置action/redraw，主循环再执行，能缩短中断处理时间并减少对其他中断的影响。

6.2 OrangeFS

磁盘布局：LBA82主目录；LBA83-114为32扇区数据池；LBA115为备份目录。目录扇区包含ORF8魔数、32位位图、7个24 B目录项、版本号和XOR校验。每个目录项记录名称、大小、起始LBA、扇区数、标志和16位数据累加校验。

Shell/文件管理器 → 文件名查目录项 → start LBA + sectors → ATA读写 → 数据校验 → 返回内容

问：为什么文件必须连续存放？

答：目录项只保存起始LBA和扇区数，没有FAT链或索引块，所以1或2个数据扇区必须连续。

问：校验和恢复有什么区别？

答：校验负责发现错误；恢复需要有效副本。OrangeFS能用备份目录恢复主目录，但文件内容只有校验没有副本，所以内容损坏只能检测。

问：为什么先写备份目录再写主目录？

答：若两次写入之间中断，较新的备份仍可能有效；重启时通过校验和版本号选择它恢复主目录。它是基础恢复，不等同于日志事务。

6.3 Shell、监视器与文本桌面

Shell是Ring0中的统一交互模块，不是用户态程序。监视器显示累计运行状态；GUI是VGA文本模式状态机，不是像素级窗口系统。键盘IRQ按Tetris→监视器→UI→Shell的顺序询问，EAX=1表示当前模块已消费按键。

6.4 selftest与QEMU回归

selftest是内核白盒快速检查；make

test-release是宿主机驱动QEMU的端到端回归。sendkey经过真实IRQ1输入路径，0xE9

debugcon收集日志，-snapshot防止测试污染基础镜像。16项测试全部通过才形成发布门禁。

第七部分 课程高频短问短答

问：操作系统为什么需要内核态和用户态？

答：为了限制普通程序的权限。用户程序不能直接执行特权指令或访问内核页，只能通过受控系统调用请求服务，从而降低单个程序破坏整个系统的风险。

问：保护模式比实模式多了什么？

答：保护模式提供描述符、特权级、32位寻址、异常和分页等保护机制；实模式主要使用段偏移形成20位地址，没有同等级的隔离。

问：为什么Boot loader不能直接完成所有初始化？

答：引导扇区只有512

B且初始处于16位实模式，空间和环境有限，因此分阶段由Loader装载较大内核并完成模式切换。

问：中断、异常和系统调用有什么共同点与区别？

答：它们最终都通过IDT进入处理程序；硬件中断来自外设，异常由CPU执行错误产生，系统调用是用户程序主动的软件中断。

问：interrupt gate为什么会清IF？

答：进入处理程序时暂时屏蔽可屏蔽中断，避免普通入口在尚未保存完整现场时被嵌套；需要时内核可显式sti。

问：iretd和ret有什么区别？

答：iretd恢复EIP、CS、EFLAGS，跨特权级时还恢复ESP和SS；ret只完成普通函数或过程返回。

问：PCB为什么只保存ESP也能调度？

答：其他寄存器已经按约定压在任务内核栈中，保存ESP就保留了整份现场的位置。PCB还记录PID和状态等管理信息。

问：线程和进程在本项目中怎样理解？

答：项目更接近固定任务加一个受保护用户任务的教学模型。它展示PCB、调度和用户空间，但没有现代进程的独立完整地址空间、句柄表和父子关系。

问：为什么内核页故障要停机？

答：Ring0故障说明内核或关键数据结构可能损坏，继续执行可能扩大破坏；教学系统选择打印诊断并停止。

问：内存泄漏怎样验证？

答：在用户任务前、运行中、结束后读取used pages，预期N、N+3、N；自动回归会检查这一关系。

问：内部碎片和外部碎片分别是什么？

答：内部碎片是已分配单元内部未使用空间，例如只用少量字节却占4 KiB页；外部碎片是空闲空间分散，OrangeFS可能有两个空闲扇区却不连续。

问：文件系统为什么需要缓存或缓冲区？

答：设备按512 B扇区传输，文件却按实际字节操作，内存缓冲区负责整扇区搬运、拼接、修改和校验。

问：为什么touch不立即占数据扇区？

答：它采用延迟分配，只创建空目录项；首次write时才根据内容申请1或2个扇区，减少空文件占用。

问：单体内核与微内核差异？

答：单体内核把主要服务放在内核地址空间，调用直接但故障隔离弱；微内核把更多服务移到用户空间，通过IPC协作，隔离更强但结构与通信更复杂。

问：如何证明系统不是一个普通程序？

答：它拥有自己的引导扇区，从BIOS取得控制权，自行建立GDT、IDT、分页和驱动，直接处理中断和硬件端口，不依赖宿主操作系统提供运行时。

问：为什么测试使用-snapshot？

答：将测试写入保存在临时覆盖层，不永久修改orange.img，保证各次测试可重复且互不污染。

问：项目最完整的一条执行链是什么？

答：exec demo.oex：键盘→Shell→OrangeFS/ATA→OEX2解析→物理页与分页→PID4调度→Ring3→int0x80→退出与资源回收。

问：项目最大的局限是什么？

答：它是固定任务数、固定用户虚拟地址、单层小型文件系统和VGA文本UI的教学系统；没有完整每进程地址空间、阻塞调度、日志文件系统、网络和硬件兼容层。

答辩收束：优点、局限与改进

三个核心优点

- 完整：从BIOS引导到Ring3、文件系统和应用形成闭环。
- 可观察：Shell、监视器和GUI让底层状态可以直接展示。
- 可验证：运行时自检加16项QEMU回归覆盖正常与故障路径。

局限的标准说法

OrangeOS定位为教学型系统，因此主动采用固定容量和直接结构换取可理解性。它尚未实现每进程独立页目录、阻塞/唤醒与优先级调度、文件日志事务、非连续文件块、权限系统、网络栈和像素级窗口系统。回答局限时应同时给出改进方向，不要把教学简化说成功能错误。

改进路线

独立CR3地址空间 → 用户代码页只读与栈不可执行 → 阻塞队列和优先级调度 → ATA错误返回与文件写入回滚 → CRC32和日志 → 多级目录与非连续块索引 → 结构化测试事件和CI。

最后一句

OrangeOS的价值不在于功能规模，而在于把操作系统课程中的启动、资源管理、保护、抽象、交互和可靠性验证，从独立知识点实现成了一条能够真实运行的完整执行链。

现场应急

- 命令输错：直接说明输入错误并重新输入，不要慌乱解释。
- GUI卡住：按Esc或Q；仍无响应就回到Shell演示selftest和exec主线。
- 时间不足：保留selftest、exec demo.oex、cat defense.txt和16/16结论，删除monitor停留和GUI深入操作。
- 被打断提问：先回答问题，再用一句“我继续展示这条执行链”回到当前步骤。